

Implementación de algoritmos de álgebra lineal numérica

Rodolfo M. Turpo R.

12 de diciembre de 2025

Índice

1. Back substitution & forward substitution	2
2. Matriz de Householder	2
3. Factorización QR	3
3.1. Solución de sistemas	4
4. Factorización de cholesky	4
4.1. Solución de sistemas	5
5. minimos cuadrados	5
5.1. Solución de sistemas	5
6. Métodos iterativos	5
6.1. Jacobi	5
6.2. Gauss-Jacobi	5
7. Método de las potencias	5
8. Singular Value Decomposition	6
8.1. Solución de sistemas con SVD	6

1. Back substitution & forward substitution

```
1 def forward_substitution(A, b):
2     n = b.shape[0]
3     y = np.zeros_like(b, dtype='float')
4     y[0] = b[0] / A[0, 0]
5     for i in range(n):
6         suma=0
7         for j in range(i):
8             suma=suma+A[i,j]*y[j]
9         y[i]=(b[i]-suma)/A[i,i]
10    return y
11 def backward_substitution(A, b):
12     n = b.shape[0]
13     x = np.zeros_like(b, dtype='float')
14     x[n-1] = b[n-1] / A[n-1, n-1]
15     for i in range(n-2, -1, -1):
16         suma=0
17         for j in range(i+1, n):
18             suma=suma+A[i,j]*x[j]
19         x[i]=(b[i]-suma)/A[i,i]
20    return x
```

2. Matriz de Householder

La importancia de las matrices de Householder es que pueden crear ceros en las columnas de una matriz por debajo de la entrada de la diagonal principal de A .

Definición 2.1

Una matriz de la forma

$$H = I - \frac{2uu^T}{u^T u}$$

donde u es un vector no nulo, es llamada **matriz de Householder**.

Lema 2.1

Dado un vector no nulo $x \neq e_1$, existe una matriz de Householder H tal que Hx es múltiplo de e_1

Demostración:

Definamos $H = I - \frac{2uu^T}{u^T u}$ con $u = x + \text{sign}(x_1)\|x\|_2 e_1$. Entonces, multiplicando, se puede ver que Hx es un múltiplo de e_1 ■

Algorithm 1: Producto de un vector con una matriz de Householder (Hx)

Entrada: Vector $u \in \mathbb{R}^n$ que define la matriz de Householder H , vector $x \in \mathbb{R}^n$.

Salida : Vector x sobrescrito con el producto Hx .

// Paso 1: Calcular el factor de escala

$$\beta \leftarrow \frac{2}{u^T u}$$

// Paso 2: Calcular el producto escalar $s = u^T x$

$$s \leftarrow \sum_{i=1}^n u_i x_i$$

// Paso 3: Actualizar β con el producto escalar

$$\beta \leftarrow \beta \cdot s$$

// Paso 4: Actualizar el vector x

for $i = 1$ **to** n **do**

$$\quad \lfloor \quad x_i \leftarrow x_i - \beta u_i$$

A continuación se presenta la implementación del algoritmo:

```

1 import numpy as np
2 import pandas as pd
3 np.set_printoptions(formatter={'float': lambda x: f"{x:.4e}"})
4 def HouseHolder(x,n):
5     if np.sign(x[[0]])==0:
6         u=x+np.linalg.norm(x,2)*np.eye(n)[:,[0]]
7         H=np.eye(n)-2*u@u.T/(u.T@u)
8     else:
9         u=x+np.sign(x[[0]])*np.linalg.norm(x,2)*np.eye(n)[:,[0]]
10        H=np.eye(n)-2*u@u.T/(u.T@u)
11    return H
12 if __name__ == "__main__":
13     # Un ejemplo pequeno para probar que el script funciona solo
14     x=np.array([[ -1],[0],[3],[ -2]])
15     n=4
16     H=HouseHolder(x,n)
17     print('x\n',x)
18     print('-----')
19     print('H\n',H)
20     print('-----')
21     print('H*x\n',H@x)

```

3. Factorización QR

La descomposición QR se trata de descomponer una matriz A en dos matrices de nombre Q y R

$$A = QR$$

donde Q es una matriz ortogonal unitaria y R es una matriz rectangular

A continuación se presenta la implementación del algoritmo:

```

1 import numpy as np
2 import pandas as pd
3 np.set_printoptions(formatter={'float': lambda x: f"{x:.4e}"})
4
5 def HouseHolder(x,n):
6     if np.sign(x[[0]])==0:
7         u=x+np.linalg.norm(x,2)*np.eye(n)[:,[0]]
8         H=np.eye(n)-2*u@u.T/(u.T@u)
9     else:
10        u=x+np.sign(x[[0]])*np.linalg.norm(x,2)*np.eye(n)[:,[0]]
11        H=np.eye(n)-2*u@u.T/(u.T@u)
12    return H
13 def fact_QR(A):
14     n=A.shape[0]
15     Q=np.eye(n)
16     for i in range(n-1):
17         x=A[i:,[i]]
18         H0=HouseHolder(x,n-i)
19         H=np.eye(n)
20         H[i:,i:]=H0
21         Q=Q@H
22     R=Q.T@A
23     return Q,R
24
25 if __name__ == "__main__":
26     # Un ejemplo pequeno para probar que el script funciona solo
27     A=np.array([[ -1,4,2,0],[0,3,1,-2],[3,1,5,1],[ -2,0,1,4]])
28     [Q,R]=fact_QR(A)
29     print('Matriz original\n',A)
30     print('Q\n',Q)
31     print('-----')

```

```

32 print('R\n',R)
33 print('-----')
34 print('Matriz Q*R\n',Q@R)

```

3.1. Solución de sistemas

$$\begin{aligned}
 Ax &= b \\
 QRx &= b \\
 Qy &= b \\
 y &= Q^T b
 \end{aligned}$$

4. Factorización de cholesky

Para

$$A = HH^T$$

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} = \begin{bmatrix} h_{11} & 0 & \cdots & 0 \\ h_{21} & h_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ h_{n1} & h_{n2} & \cdots & h_{nn} \end{bmatrix} \begin{bmatrix} h_{11} & h_{21} & \cdots & h_{n1} \\ 0 & h_{22} & \cdots & h_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & h_{nn} \end{bmatrix}$$

Tenemos:

- $h_{11} = \sqrt{a_{11}}$
- $h_{i1} = \frac{a_{i1}}{h_{11}}, \quad i = 2, 3, \dots, n$
- $\sum_{k=1}^i h_{ik}^2 = a_{ii}$
- $a_{ij} = \sum_{k=1}^j h_{ik} h_{kj}, \quad j < i$

Algorithm 2: Algoritmo de Factorización de Cholesky

Entrada: Una matriz $A \in \mathbb{R}^{n \times n}$ simétrica definida positiva.

Salida : Factor de Cholesky H (matriz triangular superior).

// El algoritmo construye H fila por fila y sobrescribe la parte superior de A

for $k = 1, 2, \dots, n$ **do**

for $i = 1, 2, \dots, k - 1$ **do**

$$\left[\begin{array}{l} a_{ki} \equiv h_{ki} = \frac{1}{h_{ii}} \left(a_{ki} - \sum_{j=1}^{i-1} h_{ij} h_{kj} \right) \end{array} \right]$$

 // Cálculo del elemento de la diagonal

$$\left[\begin{array}{l} a_{kk} \equiv h_{kk} = \sqrt{a_{kk} - \sum_{j=1}^{k-1} h_{kj}^2} \end{array} \right]$$

A continuación se presenta la implementación del algoritmo:

```

1 import numpy as np
2 import pandas as pd
3 np.set_printoptions(formatter={'float': lambda x: f"{x:.4e}"})
4 def cholesky(A):
5     n = A.shape[0]
6
7     # Verificar simetria
8     if not np.allclose(A, A.T):
9         raise ValueError('La matriz no es simetrica')
10
11     # Verificar definida positiva
12     if np.min(np.linalg.eigvals(A)) <= 0:
13         raise ValueError('La matriz no es positiva definida')

```

```

14
15     H = np.zeros((n,n))
16
17     for k in range(n):
18         for i in range(k):
19             suma1 = sum(H[i,j]*H[k,j] for j in range(i))
20             H[k,i] = (A[k,i] - suma1) / H[i,i]
21
22             suma2 = sum(H[k,j]**2 for j in range(k))
23             H[k,k] = np.sqrt(A[k,k] - suma2)
24
25     return H
26 if __name__ == "__main__":
27     # Un ejemplo pequeno para probar que el script funciona solo
28     A = np.array([[4,1,1],
29                   [1,3,0],
30                   [1,0,2]], float)
31     H=cholesky(A)
32     print('A\n',A)
33     print('matriz H\n',H)
34     print('Matriz H.T\n',H.T)
35     print('A=H*H.T\n',H@H.T)

```

4.1. Solución de sistemas

5. minimos cuadrados

5.1. Solución de sistemas

6. Métodos iterativos

6.1. Jacobi

6.2. Gauss-Jacobi

7. Método de las potencias

El método de las Potencias se usa frecuentemente para hallar el autovalor dominante de una matriz. Se llama método de las potencias, porque implícitamente se construye a través de las potencias de A . Sean los autovalores $\lambda_1, \lambda_2, \dots, \lambda_n$ de A tal que

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n|$$

esto es, λ_1 es el autovalor dominante de A . Sea v_1 el correspondiente autovector asociado a λ_1 . Si $\max(g)$ denota el elemento de módulo máximo del vector g . Asumamos que A es diagonalizable.

Algorithm 3: Método de las Potencias

Paso 1. Elegir x_0

Paso 2. **for** $k = 1, 2, 3, \dots$ *hasta convergencia* **do**

$$\left| \begin{array}{l} \hat{x}_k = Ax_{k-1} \\ x_k = \frac{\hat{x}_k}{\max \hat{x}_k} \end{array} \right.$$

A continuación se presenta la implementación del algoritmo:

```

1 import numpy as np
2 import pandas as pd
3 np.set_printoptions(formatter={'float': lambda x: f"{x:.4e}"})
4
5 #en tp se indica si se quiere el eige-value maximo o minimo con 'max' o 'min'
6 def metodo_potencias(A,x0,tol,tp='max'):
7     err=np.inf

```

```

8   if tp=='min':
9       if np.min(np.linalg.eig(A)[0])<=0:
10          raise ValueError("La matriz no es positiva definida")
11      A=np.linalg.inv(A)
12      while err>tol:
13          xnp=A@x0
14          xn=xnp/np.max(xnp)
15          err= np.linalg.norm(xn-x0,2)
16          x0=xn
17      else:
18          while err>tol:
19              xnp=A@x0
20              xn=xnp/np.max(xnp)
21              err= np.linalg.norm(xn-x0,2)
22              x0=xn
23      return np.max(xnp)
24  if __name__ == "__main__":
25      # Un ejemplo pequeno para probar que el script funciona solo
26      A = np.array([
27          [2, 1, 0],
28          [1, 3, 1],
29          [0, 1, 4]
30      ], dtype=float)
31      x0=np.array([[1],[1],[1]])
32      tol=1e-4
33      eig1=metodo_potencias(A,x0,tol)
34      eig2=metodo_potencias(A,x0,tol,'min')
35      print('eigen-value_maximo:\n',eig1)
36      print('eigen-value_minimo:\n',1/eig2)
37      print('-----')
38      print('eigen-values con numpy:\n',np.linalg.eig(A)[0])

```

8. Singular Value Descomposition

8.1. Solución de sistemas con SVD